황보민

원스토어 클라이언트개발팀 기술 리드, 조직용 AI 하네스 구축, 웹, 앱 개발
010-4354-1115, 1stpie@naver.com, 경기 안양시

소개

원스토어 클라이언트개발팀의 기술 리드이자 16년 차 프론트엔드(웹, 앱) 개발자입니다.

2026년 1월부터 조직을 위한 AI 하네스를 설계하고 운영해, AI 도구를 쓰지 않던 팀원까지 하나의 AI 개발 환경에 포함시켰습니다. 개발자에게는 기획 문서를 바탕으로 Jira 일감이 자동으로 만들어지고, 에이전트가 요구사항 분석부터 구현, 리뷰, 검증까지 거쳐 완료 보고서를 제출합니다. PM은 에이전트가 남긴 기록으로 진행 상황과 문제를 파악해 해결합니다. 룰, 스킬, 에이전트, 훅 같은 하네스 구성과 루프 엔지니어링으로 이 흐름을 자동화하고, 하네스가 스스로 개선되게 했습니다. 산출물은 로컬 Markdown 지식베이스(LLM Wiki)로 관리해 신뢰도와 정확도를 높였습니다.

기술 리드로서 Clean Architecture 기반 앱 구조를 설계해 팀 표준으로 합의했습니다. 서비스를 멈추지 않고 국내 원스토어 앱의 전 계층을 리팩토링하고, 글로벌 원스토어 앱을 새 구조로 개발했습니다.

앱마켓, 런처 앱을 개발해 출시했고, 서비스 중인 항공 앱을 개선, 고도화했습니다. 제주항공 모바일 예약 웹을 고도화했고, jQuery 기반 프론트 프레임워크, Eclipse 저작 도구, 개발사를 위한 SDK와 라이브러리를 개발했습니다.

핵심 역량

역량	내용
조직을 위한 AX 환경 구축	**AI를 쓰지 않던 팀원까지 AI로 개발하게 만든 조직용 하네스를 설계, 운영 (2026.01~)** 팀 룰, 스킬, 에이전트 조합으로 업무 프로세스 자동화, 룰과 훅으로 에이전트 제어 역할별 에이전트로 컨텍스트를 격리하고, 레벨에 따라 최고 16개 축으로 코드 리뷰 플러그인 마켓플레이스를 구축해 조직용 하네스와 MCP 배포, 공유 일감 기준 워크플로(start~complete)로 분석부터 테스트, 구현, 리뷰, 실기기 검증, PR, 완료 보고까지 자동화, 단계마다 상태를 저장해 재개 매주 개인별 업무와 사용 기록으로 주간보고 자동 작성, 하네스 자가 개선, 반복 업무의 스킬 추출, 사용자의 금지 결정을 무인 세션에 반영
기술 리드	**새 구조와 기술을 어떻게 들일지 정하고, 팀이 익혀 쓰게 이끎 (2022.07~)** 앱 아키텍처와 개발 표준을 제안하고 팀 합의로 확정 팀 세션으로 반복적인 학습 진행, 샘플 프로젝트 진행, 리뷰 기준과 가이드라인으로 리뷰 문화 정착 개선 과제 정의, 작업 계획, 팀원별 진행, 마감 관리
리팩토링	**서비스를 멈추지 않고 국내 앱의 전 계층을 점진적으로 전환 (2022.07~2025.09)** 계층별, 기능별 모듈 분리와 Clean Architecture 기반 앱 리팩토링 진행 MVC → MVVM → MVI 전환 Android 레거시 기술을 Modern Android Development 기술로 전환
웹 개발	**웹 화면, 웹 프레임워크, 웹 애플리케이션 개발** JSP, jQuery, Ajax로 웹 화면 개발, NetFUNNEL로 접속 트래픽 제어, 기기 폭 대응과 크로스 브라우징 jQuery 기반 프론트 프레임워크(위젯 컴포넌트, 이벤트 래핑)와 Eclipse 기반 저작 도구(IDE 플러그인) 개발 화면부터 서버 로직, DB까지 웹 애플리케이션 개발(PHP, JSP)
모바일 개발	**3,500만 회원이 쓰는 앱을 포함해 14년간 Android, iOS e커머스 앱 개발, 운영** 공통: e커머스 앱 핵심 기능(전시, 구매, 결제, 회원), 웹–앱 네이티브 브릿지, 신규 플랫폼 대응 Android: PackageInstaller, Foreground Service, WorkManager, AIDL로 다운로드, 설치, 자동 업데이트 개발 Android: 시스템 권한 앱(선탑재 런처) 개발, 서비스 중인 앱의 UI 전면 개편 iOS: Swift 앱 개발, App Store와 엔터프라이즈 배포
SDK 개발	**개발사와 서비스를 위한 SDK, 라이브러리 개발** Play Core 역설계 분석을 바탕으로 AAB Dynamic Delivery SDK 설계와 개발, AIDL 연동 빌드 서버용 bundletool 라이브러리, APK와 AAB 리패키징, 재서명 도구(Kotlin JAR) 개발 Android View와 Box2D 물리엔진을 연동하는 SDK 개발

1. 조직을 위한 AX 환경 구축

원스토어 클라이언트개발팀, 2026.01 ~ 현재, 설계, 구축, 운영. AI를 쓰지 않던 팀원까지 팀원 전원이 이 하네스로 개발하고, 도입 뒤 QA 검증 기간 결함은 본인 담당분 0건, 팀은 이전 대비 절반 수준으로 줄어듦

직접 설계한 하네스 구성

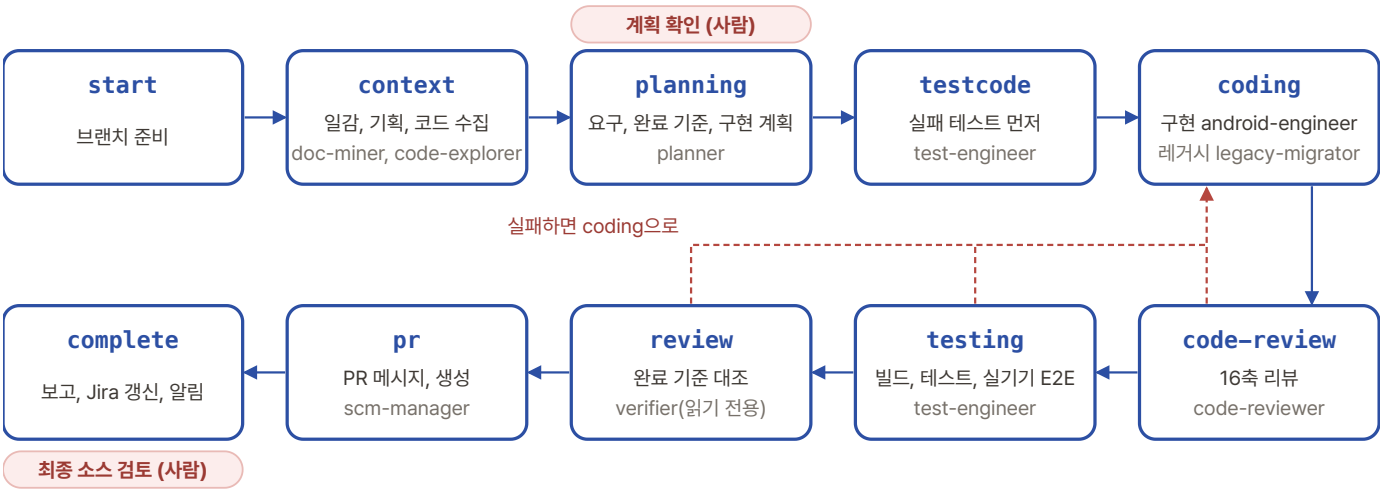
`client-dev-harness/` — 팀원 전원이 쓰는 하나의 하네스

`rules/`	11	팀 규칙, 팀 관례 / 조건부 로드 룰들
`references/`		리뷰 기준, Git, Confluence, 스킬 실행 규약, Jira 흐름, 팀 기억
`agents/`	19	`planner`, `android-architector`, `android-engineer`, `legacy-migrator`, `ux-agent`, `test-engineer`, `code-reviewer`, `doc-reviewer`, `security-reviewer`, `verifier`, `judge-assistor`, `code-explorer`, `doc-miner`, `code-optimizer`, `sonarqube-fix-team`, `scm-manager`, `task-manager`, `member-lead` 2
`procedures/`		에이전트별 절차, XML→Compose 파이프라인, 정적 분석, 리뷰 축, 실기기 테스트, 하네스 개선
`skills/`	45	`workflow-implement-feature`, `workflow-crashlytics-fix`, `workflow-create-and-review-pr`, `request-pr`, `review-code`, `deep-analysis`, `cleanup`, `scm`, `harness-improver`, `start`, `jira-subtask`, `weekly-report`, `figma-design`, `harness-audit`, `sync-harness` 등
`hooks/`	47	보호 브랜치 차단, 위험 명령 차단, 외부 전송 차단, 수정 후 동작, push 게이트 등
`scripts/`		빌드 도구, 실기기 테스트 도구, 사용 통계 전송 도구
`MCP, LSP`		사내 MCP(Jira, Confluence, Bitbucket, Teams), mobile-mcp, SonarQube 조회, Kotlin LSP

특징

- 플러그인으로 팀에 배포, 팀원 전원이 같은 환경에서 일하며, 팀원의 업무가 AI 자산이 됨
- 사용자 의사결정의 금지 사항을 저장해 무인 세션에 반영
- 주마다 사용자의 사용 데이터를 바탕으로 하네스를 개선해 팀 AI 자산화
- 워크플로 스킬을 통한 업무 파이프라인으로 일감 기준으로 업무를 시작, 각 단계마다 상태를 보고 및 저장하며, 진행 상태가 끊겨도 그 단계부터 재개
- 역할별 에이전트를 통한 컨텍스트 격리
- 사용자 사용 패턴을 통한 업무 스킬화
- Andrej Karpathy의 LLM Wiki 기반 지식베이스를 통한 LLM 산출물 관리

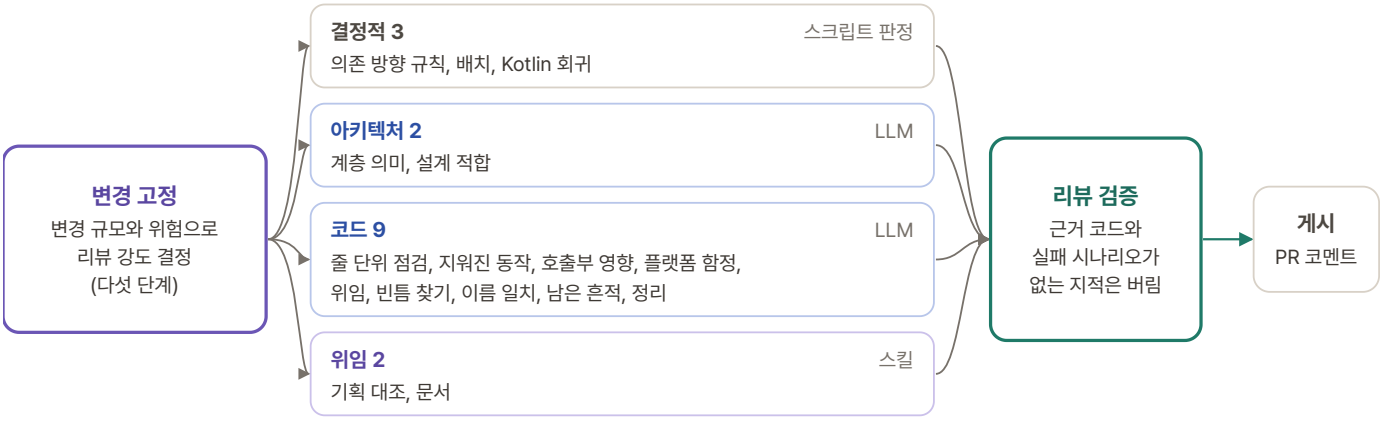
일감에서 PR까지 하나의 흐름



단계마다 진행 상태를 저장해, 멈춘 단계부터 다시 이어 감

- 기획서로 하위 일감을 만들고, 일감마다 start~complete를 진행
- 단계마다 진행 상태를 저장해, 중간에 멈춰도 그 단계부터 다시 이어 감
- 테스트, 검증은 별도의 에이전트로 컨텍스트를 격리해 작성
- 사람은 계획을 확인하고 최종 소스를 검토. 저장소에 적어 둔 금지 규칙(예: ui 모듈에서 data 직접 참조 금지)을 어겨야 하는 상황이면, 에이전트는 코드를 쓰지 않고 멈춰서 사람에게 물음

여러 축으로 보는 코드 리뷰



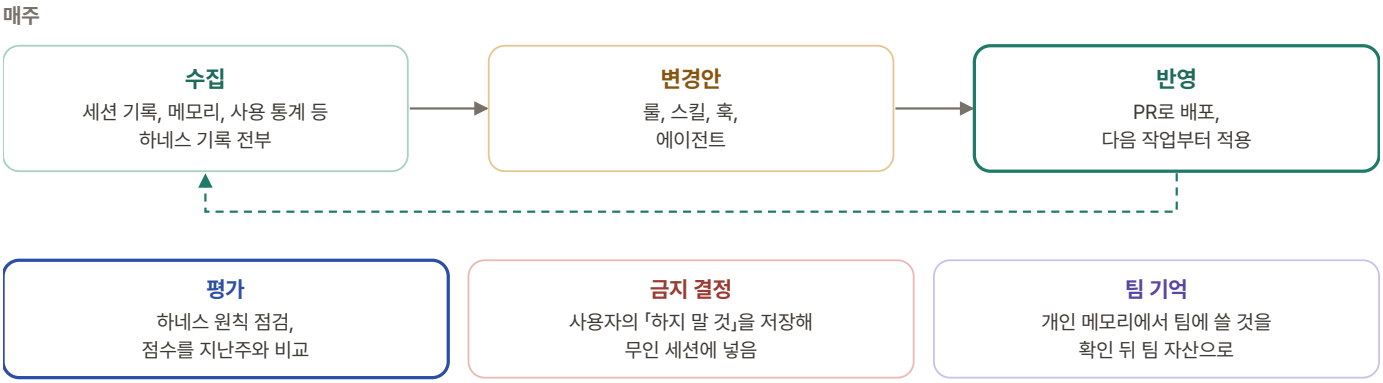
16개 축을 서브에이전트가 동시에, LSP는 참조와 호출 찾기, 컴파일 진단에 씀

- 리뷰는 레벨에 따라 최고 16개 축으로 나눠서 진행
- Kotlin LSP를 붙여 호출부 영향, 남은 흔적 같은 축이 문자열 검색 대신 참조와 호출 관계로 찾게 함
- 데일리 코드 리뷰 결과는 Confluence KPI 대시보드로 공유

빌드, 테스트 자동 검증

- 파일을 고칠 때마다 그 파일만 ktlint와 Kotlin LSP로 검사해 결과를 에이전트에게 바로 돌려줌
- 에이전트가 Gradle을 직접 실행하지 못하게 막고, 빌드 스크립트 하나가 저장소별 대기열, 같은 빌드 합치기, 오류 요약을 맡음
- 실기기 E2E는 기기를 한 번 조작하면서 화면, logcat, 서버 응답을 함께 기록하고, 통과 여부는 스크립트가 판정
- push 전에는 아키텍처 이행, 의존성 변화, 모듈 등록, 컴파일 검사를 통과해야 함

매주 점검하고 고치는 하네스



- 매주 Anthropic Claude Code 팀 Daisy Hollman의 하네스 원칙을 바탕으로 만든 점검을 돌려 점수를 기록하고 지난주와 비교, 개선을 진행
- 사용자가 정한 금지 사항은 저장해 두고 무인 세션에 넣음. 개인 메모리 중 팀에 쓸 만한 것은 확인을 거쳐 팀 자산으로 옮김

팀 전원이 쓰게 만든 배포

- 사내 플러그인 마켓플레이스에 하네스를 올림. 처음에 사용자 정보와 작업하는 앱 저장소만 입력하면 동기화 스킬이 설치와 갱신을 맡고, 머지하면 커밋 SHA 기준으로 배포
- 등록된 앱 저장소마다 앱 규칙, 권한, 검증 장치를 점검해 빠진 것만 확인을 받고 채움. 세션을 시작할 때 하루 한 번 연결과 설정을 점검해 깨진 것을 고침
- 동의한 팀원의 사용 통계로, 써야 할 때 안 쓰인 스킬과 자주 막히는 훅을 찾아 주간 개선에 넣음

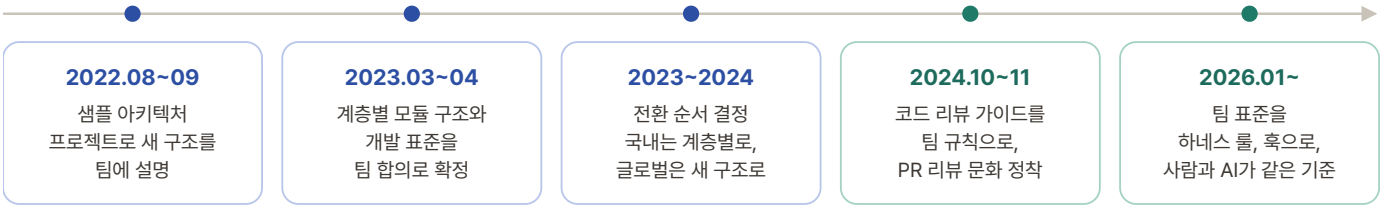
그 밖의 실적

- SonarQube 조회 MCP 서버와 팀 사용 통계 수집 서버 개발
- Andrej Karpathy의 LLM Wiki 구조(raw, wiki, schema)로 팀원별 지식베이스를 만듦. LLM이 자료를 넣고, 찾고, 점검함
- Crashlytics 원인 분석부터 승인 뒤 수정과 PR까지 자동화
- 팀 교육용 Claude Code 에이전트 가이드 12편과 기초 가이드 2편을 사내 위키에 쓰고, 회사 대상 AI 발표로 구축 경험을 공유

2. 기술 리드

새 구조를 정하고, 팀이 익혀 쓰게 만들기

원스토어, 2022.07 ~ 현재, 클라이언트개발팀(9명) 기술 리드(2023 ~)



기술 부채 과제 관리 (2022 리팩토링 단계부터 계속)
국내 앱 리팩토링, 글로벌 앱 리팩토링 등을 과제로 정의하고
작업 계획, 팀원별 진행, 릴리스 마감까지 관리

- **구조를 정하는 판단:** 앱 구조와 개발 표준을 샘플 아키텍처 프로젝트로 먼저 보여 주고 팀 합의로 확정
- **팀이 쓰게 만드는 방법:** 팀 세션을 반복하고, 코드 리뷰 가이드를 팀 규칙으로 만듦
- **부채를 끝까지 관리:** 2022년부터 기술 부채를 과제로 정의해 작업 계획, 팀원별 진행, 릴리스 마감까지 관리(국내 앱 리팩토링, 글로벌 앱 리팩토링 등)
- **사람과 AI가 같은 기준:** 계층 규칙과 리뷰 기준을 하네스 규칙으로 옮기고, 훅과 아키텍처 판정 검사로 지키게 함
- **결과:** 국내, 글로벌 두 앱이 같은 구조 표준으로 옮겨졌고, PR 리뷰가 팀 규칙으로 자리 잡음

항공 예약 파트 리더

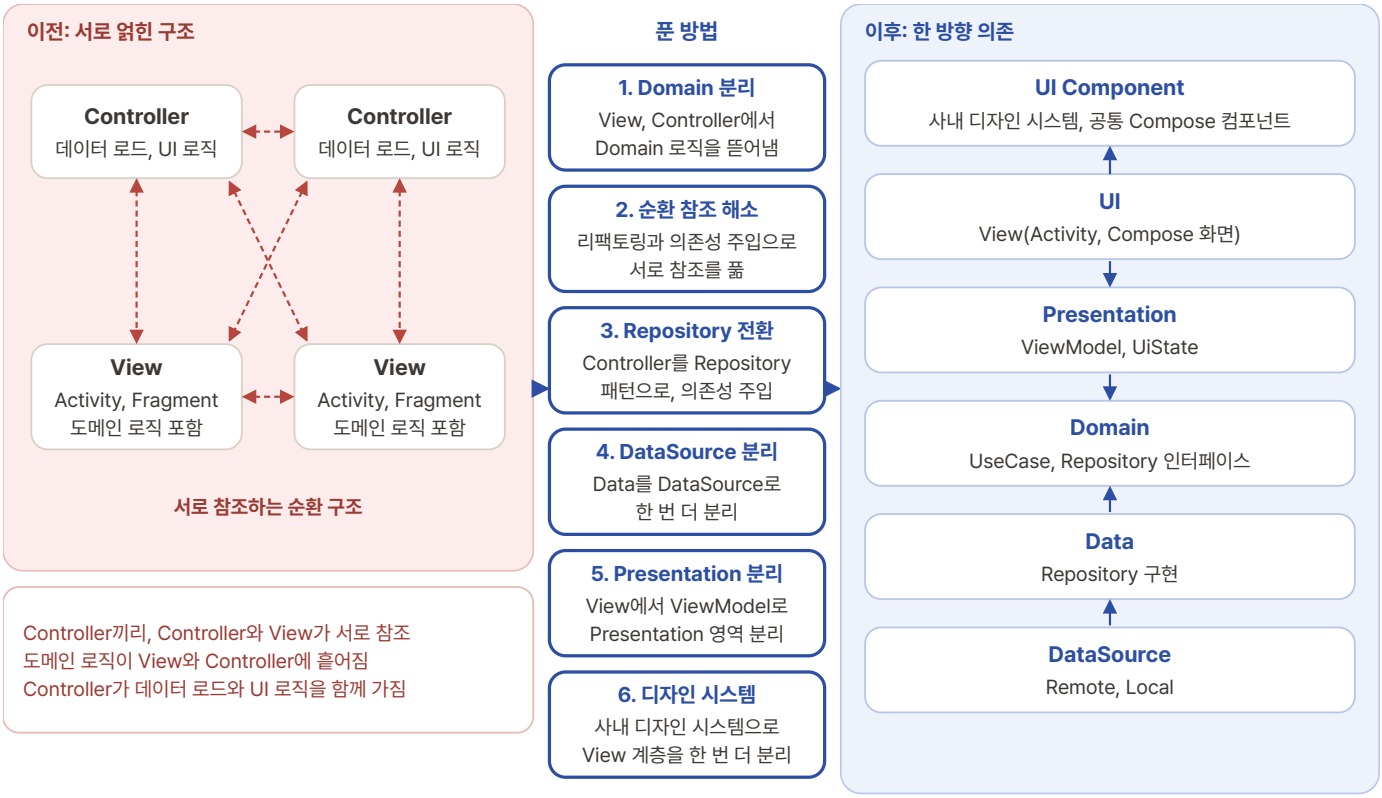
AK아이에스 (제주항공), 2019.06 ~ 2020.03

- **요건과 품질을 함께:** 고객사(제주항공) 요건을 직접 협의, 정리하고, 협력업체의 고객용, 사내 웹과 모바일 앱 코드를 리뷰, 검수하며 예약 웹, 앱 개발을 이끎

3. 리팩토링

서비스 중인 국내 원스토어 앱의 얽힌 구조 풀기

원스토어, 2022.07 ~ 2025.09, 기능 개발과 함께 계층 단위로



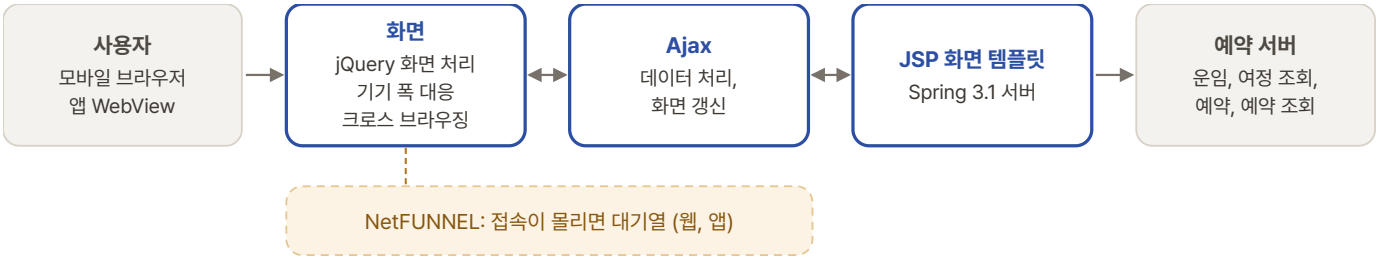
재발 방지: 계층 규칙을 하네스 룰과 훅, 의존성 검사(참조 방향, 배치, 순환)로 옮겨 사람과 AI가 같은 기준으로 코드를 씀

- **얽힌 문제:** 8년 이상 된 앱에서 Controller끼리, Controller와 View가 서로 참조하고, 도메인 로직이 View와 Controller에 흩어져 있었음. 데이터 로드를 맡은 Controller 11개에는 UI 로직까지 섞여 있었음
- **끊는 순서:** View, Controller에서 Domain 로직을 뜯어내고, Controller와 View 사이 순환 참조를 리팩토링과 의존성 주입으로 풂. Controller는 Repository 패턴으로 바꿔 의존성을 주입하고, Data는 DataSource로 한 번 더 분리. View는 ViewModel로 Presentation 영역을 나누고, 사내 디자인 시스템을 만들어 View 계층을 한 번 더 분리
- **이후 구조:** 디자인 시스템과 공통 Compose 컴포넌트를 UI Component 계층으로 두고, UI, Presentation, Domain, Data, DataSource가 한 방향으로 의존

4. 웹 개발

제주항공 모바일 예약 웹 고도화

AK아이에스, 2018.07 ~ 2020.03



- **예약 흐름 화면:** 운임, 여정 조회, 예약, 예약 조회 화면과 안내 페이지를 고도화. JSP 화면 템플릿(Spring 3.1), jQuery 화면 처리, Ajax 데이터 처리를 맡음
- **어느 기기에서나 같은 화면:** 미디어 쿼리로 폰 화면 폭별 배치를 맞추고 앱 WebView 전용 스타일을 적용. 브라우저 차이는 jQuery와 브라우저별 CSS로 보정
- **몰리는 접속 제어:** 접속이 몰리는 예약 화면에 NetFUNNEL 대기열을 웹과 앱에 적용

다빈치: HTML5 프론트 프레임워크와 저작 도구

이노에이스, 2011.07 ~ 2013.06



- **프레임워크 설계:** jQuery 위에 위젯 컴포넌트와 컴포넌트 라이프사이클에 맞춘 이벤트 래핑을 얹은 HTML5 프론트 프레임워크
- **저작 도구와 빌드:** 그 프레임워크로 화면을 그리는 Eclipse 기반 WYSIWYG 저작 도구(IDE 플러그인)와 웹앱을 Android 앱으로 만드는 하이브리드 빌드

공군 전산실 웹 시스템

공군 (병역), 2004.10 ~ 2007.01



웹 서버(Tomcat, Apache)와 Oracle, MySQL DB 운영도 맡음

- **화면부터 DB까지:** 인트라넷 포털, 장병 휴가 관리, 수송 배차 관리 등 부대 업무용 웹 애플리케이션을 Struts 1, JSP, PHP로 화면부터 서버 로직, DB까지 개발하고 웹 서버, DB를 운영

5. 모바일 개발

Android, iOS 앱 14년

이노에이스, AK아이에스, 원스토어, 2010.01 ~ 현재

2010 ~ 2014

런처와 UI 엔진

폰꾸미기(선탑재, 400만 다운로드)
T간편모드(기본 탑재 런처)
Rabbit 3D 런처, MUIF 3D UI

2014 ~ 2017

앱마켓

티스토어 신규 개발, 고도화
원스토어 6.0 UI 전면 개편
전시, 상세, 검색, 선물, 구매

2018 ~ 2020

iOS 항공 앱

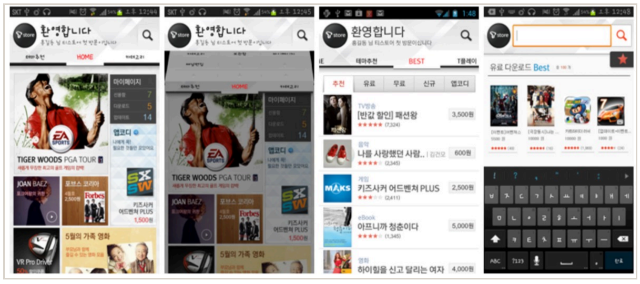
제주항공 고객 앱(Swift)
예매, 모바일 탑승권
엔터프라이즈 사내 배포

2020 ~ 현재

원스토어 국내, 글로벌

3,500만 회원 앱마켓
다운로드, 설치, 업데이트
전시, 회원, 결제, SDK

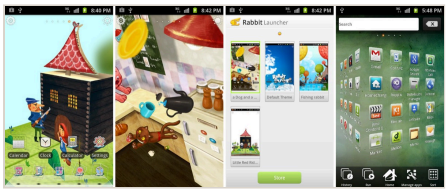
- **3,500만 회원 앱마켓:** 통신사 단말에 기본 탑재되는 원스토어 국내 앱의 다운로드, 설치, 자동 업데이트, 동적 전시와 상품 상세, 회원과 결제 연동을 담당하고, 2023년 글로벌 앱을 멀티 모듈 구조로 설계해 개발
- **플랫폼 깊이:** 시스템 권한으로 단말 설정을 제어하는 선탑재 런처(T간편모드), PackageInstaller 세션 설치, AIDL로 다른 앱과 연동하는 SDK, 릴리스마다 신규 OS 대응(Android 11 설치, Android 13 알림, 16KB Page Size)
- **iOS:** 제주항공 고객 앱을 Swift로 개발해 App Store로 배포하고, 사내 앱은 엔터프라이즈 인증서로 배포. UIWebView에서 WKWebView로 바꾸며 끊긴 웹–앱 브릿지 호출, alert, 새 창, 로그인 세션 문제를 해결



티스토어: 홈, 베스트 목록, 검색 화면



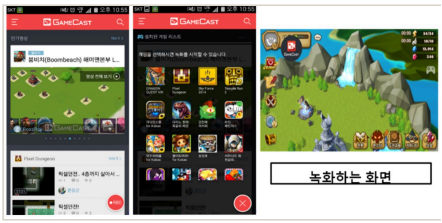
T간편모드: 홈 모드 전환, 키패드형 홈, 화면, 소리 설정, 간편 메뉴



Rabbit Launcher: 3D 테마 홈, 테마 선택, 앱 서랍



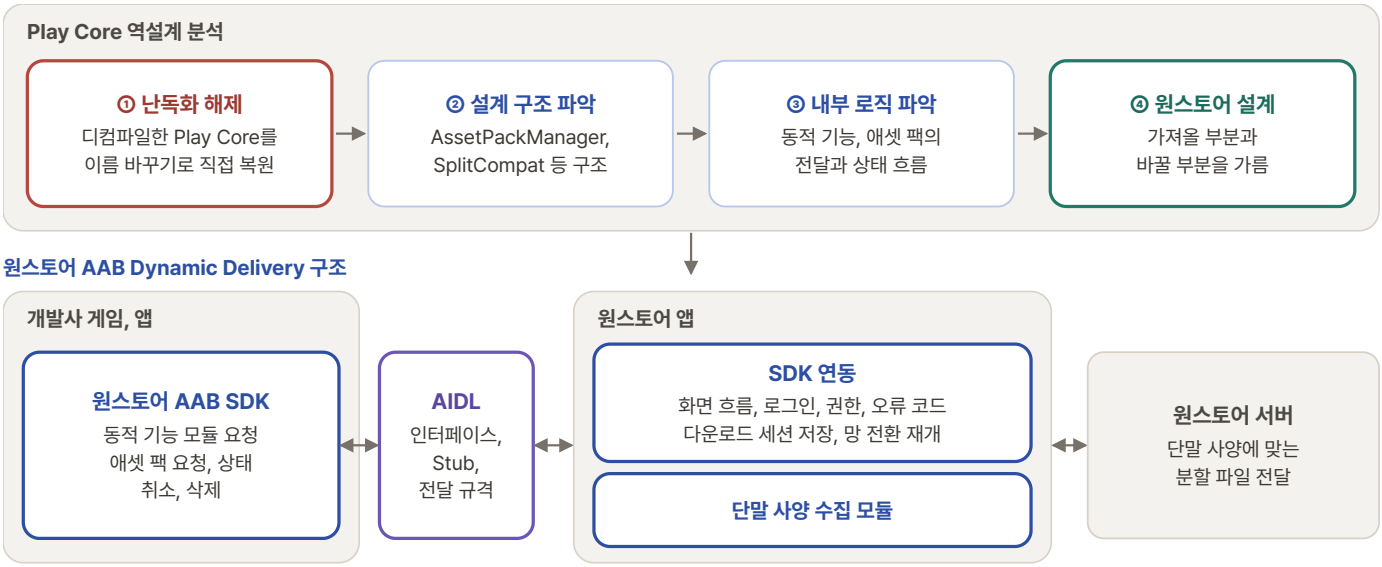
폰꾸미기: 테마 상점과 테마 꾸미기, CUKI 런처 메뉴



GameCast: 인기 영상, 녹화할 게임 목록, 게임 녹화 화면

AAB Dynamic Delivery SDK: Play Core 역설계 분석에서 설계까지

원스토어, 2020.10 ~ 2021.10



- **문제:** 원스토어는 AAB를 지원하지 않아 개발사가 동적 기능 모듈, 애셋 팩을 원스토어에서 내려받을 방법이 없었음
- **가장 어려웠던 일:** 참고할 Google Play Core는 난독화돼 있어, 디컴파일한 코드를 이름 바꾸기(rename) 등으로 직접 읽을 수 있게 복원. 그 위에서 설계 구조와 내부 기능 로직을 파악하고, 가져올 부분과 원스토어에 맞게 바꿀 부분을 가름
- **설계, 구현:** SDK 인터페이스와 원스토어 앱 사이 AIDL 규격(인터페이스, Stub, 전달 데이터)을 설계하고, 원스토어 앱 쪽 화면 흐름, 로그인, 권한, 오류 코드와 다운로드 세션 저장, 망 전환 재개를 구현. 개발사용 샘플 앱과 SDK 개발 가이드 작성

그 밖의 SDK, 라이브러리, 도구

- **원스토어용 bundletool 라이브러리:** 원스토어 앱 빌드 서버가 불러 쓰는 JAR 라이브러리로 개발
- **APK와 AAB 리패키징, 재서명 도구:** 글로벌 스토어 개발사 앱을 다시 패키징, 서명하는 Kotlin 실행 JAR. bundletool, apksig로 서명 체계 (v1~v4) 검증, 여러 서명 키 처리
- **SK Physics Android SDK (2014.07 ~ 2014.10):** ForceLayout 구조와 물리 스레드 기반 View 갱신을 설계해 Android View와 Box2D 물리엔진을 연동